Operations on Processes

Lecture Notes — Lecture 7

Creation, Termination, `fork/exec`

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

7.1 Who Actually Runs a Command?

Type `ls` at a shell prompt. A directory listing appears and the prompt returns. Two facts about this ordinary event are worth stating carefully. The shell did not turn into `ls` — it is still there, waiting for the next command. And `ls` did not exist as a process a moment earlier. So a new process came into being, ran, and disappeared, and the shell knew exactly when it had finished. This chapter explains how.

The short answer is that every process except the very first is created by another process. The shell creates a **child**, the child turns itself into `ls`, and the shell **waits** for it. Three system calls do the work — `fork`, `exec`, and `wait` — and a fourth, `exit`, ends the child. We follow one shell command from creation to termination as the running thread of the chapter.

The chapter answers four questions in order: how a process is created (parents, children, and the process tree); what `fork` returns and how many processes a program produces (counting forks); how a child becomes a different program (the `exec` family); and how a process ends and who cleans up after it (`exit`, `wait`, zombies, and orphans). Lecture 5 gave us the PCB and Lecture 6 moved a process between states; here we create and destroy the PCB itself.

Reading: Silberschatz et al., Ch. 3 §3.3, pp. 110–116 [3]; Stallings, Ch. 3 §3.4, p. 135 [4].

7.2 Process Creation

7.2.1 Parents, children, and the process tree

A process is not conjured by the kernel out of nothing: it is copied from a parent that already exists. When one process creates another, the creator is the parent and the new process is the child; the child may in turn create children of its own. Recording every creation as an arrow from parent to child produces a tree.

Definition (Parent, child, process tree). *A **parent process** is a process that issues a creation call; the new process is its **child**. Repeated creation forms a **process tree** rooted at the first process.*

Each process is identified by a unique integer, its **process identifier** (pid). Figure 7.1 shows a typical tree on a UNIX-like system: the first process, `init` with pid 1, has started a login program, an SSH daemon, and a scheduler daemon; each login path has started a shell; and one shell has started `ls` and `vim`. The pids are illustrative.

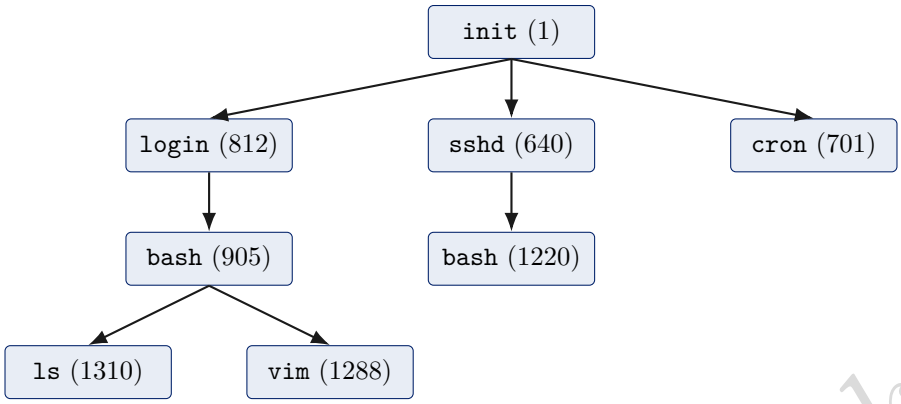


Figure 7.1: A process tree. Each arrow is one creation; pids are illustrative — cf. Silberschatz et al., Ch. 3 §3.3, pp. 110–116 [3].

7.2.2 What a parent must decide

Creating a child forces three design decisions, and every operating system answers them in some way [3]:

1. **Resources.** The child may share all of the parent's resources (open files, memory), a subset of them, or none.
2. **Execution.** The parent may continue to run concurrently with the child, or it may wait until some or all of its children terminate.
3. **Address space.** The child may be a duplicate of the parent (the same program and data), or it may have a new program loaded into it.

UNIX answers the address-space question in two separate steps: `fork` duplicates the parent, and `exec` loads a new program.

7.2.3 `fork`: one call, two returns

Definition (`fork`). *`fork()` creates a new process whose address space is a copy of the caller's. Both processes continue at the instruction after the call.*

Because both processes resume at the same point in the same code, they need a way to tell themselves apart. The only difference between them at that moment is the value `fork` returns, summarized in Table 7.1.

Who sees it	Return value of `fork()`
the child	`0`
the parent	the child's pid (a positive integer)
the parent, on failure	`-1`, and no child is created

Table 7.1: The three possible return values of `fork`.

The return value is the *only* way the two copies can distinguish themselves. A child that needs its own pid calls `getpid()`; it learns its parent's pid with `getppid()`.

Concepts That Click

Example 7.1 (The canonical `fork`/`exec`/`wait`)**.** *The following fragment has the same structure as the standard textbook example [3]:*

```
pid_t pid = fork();
if (pid < 0) {            /* error  */
    perror("fork");
    exit(1);
} else if (pid == 0) {    /* child  */
    execlp("ls", "ls", NULL);
    perror("exec");  /* on failure */
    exit(1);
} else {                  /* parent */
    wait(NULL);
    printf("Child complete\n");
}
```

Trace it step by step.

1. *`fork` executes once and returns twice: once in the parent with the child's pid, once in the child with `0`.*
2. *In the child, `pid == 0`, so the child takes the middle branch and calls `execlp`, replacing itself with the `ls` program. The `perror` and `exit(1)` that follow run only if the `exec` fails.*
3. *In the parent, `pid` is positive, so the parent takes the `else` branch and blocks in `wait`.*
4. *When `ls` finishes, `wait` returns and the parent prints `Child complete`.*

Figure 7.2 draws the same program as a timeline. The parent's line is interrupted by the `wait` box: the parent is blocked there while the child runs. The child's line begins at the `fork` point, passes through `exec("ls")`, and ends at `exit()`; the arrow back up to the parent carries the exit status and wakes the parent.

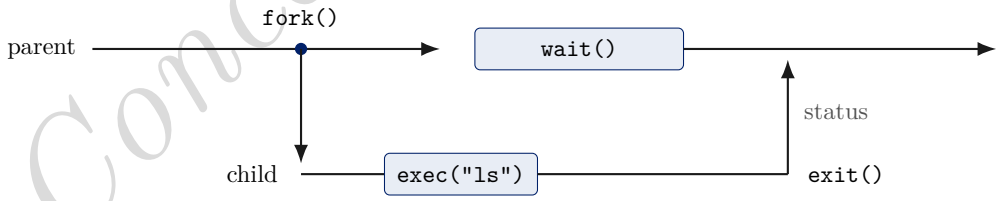


Figure 7.2: `fork`, `exec`, `exit`, and `wait` on a timeline. The parent is blocked in `wait` while the child runs; the child's `exit` wakes it and delivers a status.

7.2.4 Two copies, two variables

After `fork`, the parent and child have *separate* address spaces whose contents are initially identical. A write in one is invisible to the other.

Example 7.2 (Separate copies of a variable)**.** *Consider `int x = 10; pid = fork();` followed by:*

- *child: `x += 5; printf("child x=%d", x);`*

Concepts That Click

- *parent:* `wait(NULL); x -= 5; printf("parent x=%d", x);`

Both processes start with their own `x` *equal to 10. The child adds 5 to its copy and prints* `child x=15`*. The parent waits for the child, then subtracts 5 from* its *copy, which is still 10, and prints* `parent x=5`*. (This output was verified by running the program.)*

The variable has the same name and the same virtual address in both processes, but the two names refer to different physical copies. What *is* shared after `fork` is the set of open file descriptors: the child's descriptors point to the same open-file entries as the parent's.

A consequence follows immediately. Suppose a child wants to report the answer to a computation by storing it in a global variable that the parent reads after `wait`. That cannot work: the global lives in the child's copy of the address space, and the parent's copy still holds the old value. The child can return a small integer through its `exit` status; anything larger requires inter-process communication, the subject of Lectures 14–15.

7.3 Counting Forks

Every `fork` doubles the number of processes that execute the next line. Counting the processes a program creates is therefore a matter of drawing a tree, not of guessing.

7.3.1 Sequential forks

Example 7.3 (Three forks in a row)**.** *Count the lines printed by* `fork(); fork(); fork(); printf("hi\n");`*.*

1. *Before any fork there is one process, P.*
2. *The first* `fork` *creates C1. Now 2 processes execute the second* `fork`*.*
3. *Each of the 2 forks: P creates C2, C1 creates C3. Now 4 processes execute the third* `fork`*.*
4. *Each of the 4 forks: P creates C4, C2 creates C5, C1 creates C6, C3 creates C7. Now 8 processes execute the* `printf`*.*

So 8 lines are printed (Figure 7.3). In general, n sequential forks give 2^n *processes, of which* $2^n - 1$ *are new children. Running the program confirms 8 lines for* $n = 3$ *and 16 for* $n = 4$*.*

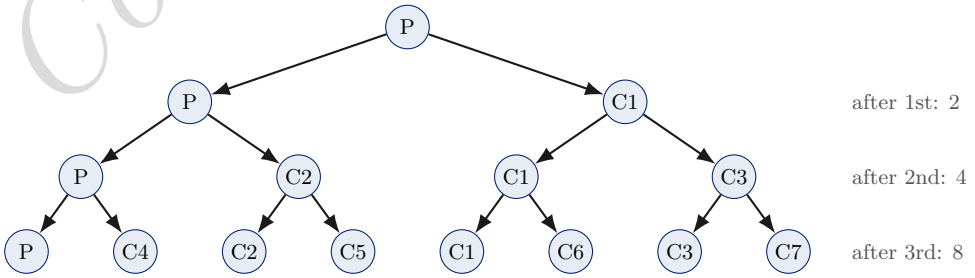


Figure 7.3: The process tree for three sequential `fork` calls. Each level doubles the number of processes: 2, then 4, then 8.

Example 7.4 (`printf` inside the loop)**.** *Count the lines printed by* `for (i = 0; i < 3; i++) { fork(); printf("A\n"); }`*. Here the print is* inside *the loop, so we count per iteration.*

Concepts That Click

1. *Iteration $i = 0$: 1 process forks into 2; each prints once: 2 lines.*
2. *Iteration $i = 1$: 2 processes fork into 4; each prints: 4 lines.*
3. *Iteration $i = 2$: 4 processes fork into 8; each prints: 8 lines.*

The total is $2 + 4 + 8 = 14$ lines of `A`, from 8 processes in total. (Verified by running it.)

7.3.2 Short-circuit operators prune the tree

In C, `a && b` evaluates `b` only if `a` is non-zero, and `a || b` evaluates `b` only if `a` is zero. Since `fork()` is non-zero in the parent and zero in the child, these operators decide *which copy* goes on to fork again.

Example 7.5 (`((fork() && fork()) || fork())`). *Count the lines printed by `(fork() && fork()) || fork(); printf("y\n");`.*

1. *First `fork`: the parent P sees a non-zero value; the child C1 sees zero.*
2. *P evaluates the second `fork`: P sees non-zero, the new child C2 sees zero. P's `&&` is true, so P skips the `||`.*
3. *C1's `&&` is already false (its first operand was zero), so C1 evaluates the third `fork`, creating C3.*
4. *C2's `&&` is false (its second operand was zero), so C2 also evaluates the third `fork`, creating C4.*

C3 and C4 see zero from the third `fork` but have nothing left to evaluate. The processes are P, C1, C2, C3, C4, so 5 lines are printed. (Verified.)

Example 7.6 (`if (fork() && fork()) fork()`). *Count the lines printed by `if (fork() && fork()) fork(); printf("x\n");`.*

1. *First `fork`: P and C1. C1 sees 0, so its condition is false and it skips the body.*
2. *P runs the second `fork`: P and C2. C2 sees 0 and skips the body.*
3. *P's condition is true, so P runs the body's `fork`, creating C3.*

The processes are P, C1, C2, C3: 4 lines. (Verified.)

7.3.3 The buffer that gets copied

Lecture 3 showed that `printf` writes into a user-space buffer and that the `write` system call comes later. `fork` copies the *whole* address space — including any bytes still sitting in that buffer.

Example 7.7 (Duplicated output). *The program*

```
printf("hello "); fork(); printf("world\n");
```

prints `hello world` twice. The string `"hello "` is still unflushed in the stdio buffer when `fork` runs, so it is duplicated into the child; each process then appends `"world\n"` and flushes its own buffer. (Verified.) The fix is to call `fflush(stdout)` before `fork`. The lesson is that `fork` copies state*, not just code.*

Concepts That Click

7.4 The exec Family

A copy of the shell is useless until it stops being the shell. `exec` throws the old program away and keeps the process.

Definition (exec). *An `exec` call replaces the calling process's memory image — code, data, heap, stack — with a new program loaded from an executable file, and starts it at its entry point.*

Several things survive the replacement. The **pid stays the same**, and so does the parent relationship. Open file descriptors stay open unless they are marked close-on-exec. What does not survive is the old code itself, which is why a successful `exec` **never returns**: the code that called it no longer exists. On failure `exec` returns `-1` and the old program continues. The Linux implementation is `execve` (Bovet & Cesati, Ch. 20, pp. 808–834 [2]).

7.4.1 Reading the exec names

The family's names are built from letters, each with a fixed meaning (Table 7.2).

Letter	Meaning
`l`	arguments as a *list*: `execl("/bin/ls", "ls", "-l", NULL)`
`v`	arguments as a *vector* (array): `execv(path, argv)`
`p`	search the `PATH` for the file: `execlp("ls", ...)`
`e`	pass an explicit *environment*: `execle`, `execve`

Table 7.2: Decoding the `exec` family names.

All of these are library wrappers; on Linux the one system call underneath is `execve`. Argument lists end with `NULL`, and by convention `argv[0]` is the program's own name.

Example 7.8 (Code after `exec`). *The child executes `execlp("echo", "echo", "from-exec", NULL); printf("never\n");` while the parent executes `wait(NULL); printf("parent\n");`.*

1. *The child's image is replaced by `echo`, which prints `from-exec` and exits.*
2. *The `printf("never")` line belonged to the old image. It is gone, so it never runs.*
3. *The parent's `wait` returns, and the parent prints `parent`.*

The output is `from-exec` followed by `parent` — two lines, never three. (Verified.)

7.4.2 Why split creation into fork and exec?

Between the two calls, the child is still running *the shell's code*, but in its own process. That gap is where it can rearrange its own environment before the new program starts.

Example 7.9 (The shell running `ls > out.txt`).

1. *The shell calls `fork`.*
2. *The child opens the target: `fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC)`.*
3. *The child calls `dup2(fd, 1)` and then `close(fd)`; standard output (descriptor 1) now refers to the file.*

Concepts That Click

4. *The child calls* `execlp("ls", "ls", NULL)`. `ls` *writes to descriptor 1 as usual and never knows about the file.*

5. *The shell calls* `wait`.

Redirection, pipes, and changed working directories all happen in this gap.

7.4.3 Is copying the whole address space wasteful?

A child that calls `exec` immediately discards the copy that `fork` just made, so a literal copy would be wasted work. Modern kernels avoid it with **copy-on-write**: parent and child share the pages read-only, and a page is copied only when one of them writes to it. `fork` then costs roughly a copy of the page tables and the PCB, not of the memory. Linux also provides `vfork` and `clone` as variants of process creation (Bovet & Cesati, Ch. 3, pp. 79–130 [2]). Copy-on-write is a virtual-memory mechanism, and we return to it in Lecture 26.

7.4.4 Another design: one-step creation

UNIX is not the only answer. Table 7.3 contrasts it with the Windows design [3].

UNIX: two steps	**Windows: one step**
`fork` duplicates, then `exec` loads	`CreateProcess` loads a named program into a new process
Child customizes itself between the calls	Customization passed as parameters to the single call
Parent waits with `wait`	Parent waits with `WaitForSingleObject`

Table 7.3: Two-step versus one-step process creation.

Both designs answer the same three parental decisions (resources, execution, address space); they differ in *where* the answers are supplied.

7.5 Process Termination

Creation is half the life cycle. When a process ends, something must collect what it leaves behind.

7.5.1 Normal termination: `exit` and `wait`

A process terminates when it finishes its last statement or calls `exit(status)`. The kernel then releases its memory, open files, and other resources, but keeps the small integer `status` for the parent. The parent collects it with `wait`, which returns the pid of the terminated child; `waitpid` waits for one named child, whereas `wait` waits for any.

Example 7.10 (Collecting an exit status)**.** *The child calls* `exit(7)`. *The parent calls* `waitpid(pid, &st, 0)`. *When it returns,* `WIFEXITED(st)` *is true (the child exited normally) and* `WEXITSTATUS(st)` *is 7. (Verified.)*

Concepts That Click

7.5.2 When a parent terminates a child

A parent may also end a child with a system call — `kill` on UNIX, `TerminateProcess` on Windows. Only a parent, or a suitably privileged user, may do so. Typical reasons [3] are that the child has exceeded its allocated resources, that the task assigned to it is no longer required, or that the parent is exiting and the system does not allow a child to continue without its parent.

Definition (Cascading termination). *On systems that do not allow a child to outlive its parent, terminating a parent terminates all of its descendants. This* ***cascading termination*** *is initiated by the operating system.*

7.5.3 Zombies and orphans

The gap between a child's `exit` and its parent's `wait` creates two bookkeeping states. Their treatment here is the standard UNIX one (Bovet & Cesati, Ch. 3, destroying processes, pp. 79–130 [2]).

Definition (Zombie). *A* ***zombie*** *is a process that has terminated but whose parent has not yet called* ***`wait`****. Its resources are freed, but its PCB entry (pid and exit status) remains so that the parent can collect it.*

Definition (Orphan). *An* ***orphan*** *is a process whose parent terminated without waiting for it. UNIX re-parents an orphan to* ***`init`*** *(pid 1), which calls* ***`wait`*** *periodically.*

Every child is briefly a zombie, between its `exit` and its parent's `wait`; that is normal. The problem is a parent that never waits. Many zombies exhaust the pid table even though they use no memory. A zombie is a bookkeeping state, not a running program. Figure 7.4 shows both paths: the solid path is the normal case (`fork` creates, `exit` makes a zombie, the parent's `wait` reaps it); the dashed path is the case in which the parent exits first, the child becomes an orphan of `init`, and `init`'s `wait` reaps it.

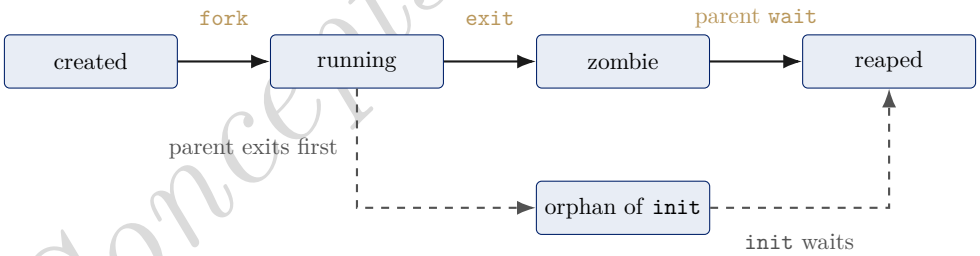


Figure 7.4: The process life cycle at the boundary. Solid: the normal path. Dashed: the parent exits first and `init` adopts and reaps the child.

Example 7.11 (A server that never waits)**.** *A server forks one child per request and never calls* ***`wait`****; each child finishes in milliseconds. Every finished child becomes a zombie, so its pid and status stay in the process table. Memory is released, but after enough requests the pids run out. The fix is to reap the children — call* ***`wait`*** *or* ***`waitpid`*** *as they finish, for example in a loop with* ***`WNOHANG`****.*

7.5.4 Putting it together: the shell loop

Example 7.12 (The shell loop)**.** *A shell repeats five steps.*

Concepts That Click

1. *Print the prompt and read a command line, e.g.* `ls -l`.
2. *Call* `pid = fork()`.
3. *In the child (*`pid == 0`*): set up any redirection, then call* `execvp("ls", argv)`*; if* `exec` *returns, print an error and call* `exit(127)`.
4. *In the parent: call* `waitpid(pid, &status, 0)` *— unless the line ended with* `&`*, in which case return to step 1 immediately.*
5. *Repeat.*

For each foreground command the process-control calls are `fork` *(shell),* `execvp` *(child),* `exit` *(child), and* `wait` *(shell): 4 calls, all in the process-control category of Lecture 3.*

7.6 Common Misconceptions

Table 7.4 pairs five common errors with their corrections.

Wrong	Right
"`fork` returns the child's pid to the child."	The child gets `0`; the parent gets the child's pid.
"Parent and child share variables."	They have separate copies; only descriptors are shared.
"`exec` creates a new process."	`exec` replaces the image; the pid is unchanged.
"Code after a successful `exec` runs when it finishes."	A successful `exec` never returns.
"A zombie is still using CPU and memory."	A zombie holds only its pid and exit status.

Table 7.4: Common misconceptions about process operations.

7.7 Summary

- Processes form a **tree**: every process but the first has a parent.
- `fork` returns **twice**: `0` in the child, the child's pid in the parent, `-1` on failure.
- n sequential forks give 2^n processes; short-circuit operators and loops change the count, so draw the tree.
- `exec` keeps the process and replaces the program; it returns only on failure.
- `exit` delivers a status and `wait` collects it. An unreaped child is a **zombie**; a parentless one is an **orphan** adopted by `init`.

Concepts That Click

7.8 Looking Ahead and Further Reading

This chapter covered how processes are born, transformed, and reaped. The next lecture asks what happens once many processes are ready: which one gets the CPU? It introduces the scheduling criteria (TAT, WT, RT) and the first two policies, FCFS and SJF. Read Silberschatz et al., Ch. 5 §5.1–5.3 [3], and Stallings, Ch. 9 §9.1–9.2 [4].

For this chapter's material:

- Silberschatz et al., 8e, Ch. 3 §3.3 (operations on processes: creation, termination, `fork`/`exec`, pp. 110–116) [3].
- Stallings, 6e, Ch. 3 §3.4 (process control, p. 135) [4].
- Bovet & Cesati, 3e, Ch. 3 (creating and destroying processes, pp. 79–130) and Ch. 20 (program execution, pp. 808–834) [2].
- Bach, *The Design of the UNIX Operating System* — chapter-level reference for the UNIX process view [1].

Page numbers are verified via each book's `index.md` under the course's supporting-books directory; Bach is not indexed.

Concepts That Click

Self-Check

Exercises

1. State the three possible return values of `fork` and who sees each. Why is the return value the only way the two copies can tell themselves apart?
2. How many lines does
   `for (i = 0; i < 4; i++) fork(); printf("p\n");`
   print, and how many new processes are created?
3. How many lines does
   `if (fork() == 0) fork(); else { fork(); fork(); } printf("z\n");`
   print? Draw the tree.
4. A child calls `execlp` on a program name that does not exist. What happens to the code after the call?
5. Distinguish a zombie from an orphan, and state who eventually reaps each.

Solutions

1. The child sees `0`; the parent sees the child's positive pid; on failure the parent sees `-1` and no child exists. After a successful `fork` the two processes run the same code from the same point with identical memory, so the only datum that differs is the value the call returned.
2. The `printf` is after the loop, so we count processes. Four sequential forks give $2^4 = 16$ processes, so 16 lines are printed, and $16 - 1 = 15$ processes are new children. (Verified: 16 lines.)
3. First `fork`: P and C1. C1 sees 0 and takes the `if` branch, forking once: C1 and C2. P takes the `else` branch and forks twice: after the first, P and C3; both then execute the second `fork`, giving P, C3, C4, C5. The processes reaching `printf` are C1, C2, P, C3, C4, C5: 6 lines. (Verified: 6.)

4. `execlp` fails and returns `-1`; the old image is intact, so the code after the call — normally a `perror` and `exit(1)` — runs.
5. A zombie has terminated but its parent has not yet called `wait`; its parent reaps it with `wait`. An orphan is still running (or will terminate) after its parent exited without waiting; it is re-parented to `init`, which reaps it.

References

[1] Maurice J. Bach. *The Design of the UNIX Operating System*. Prentice Hall of India, 1986.

[2] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition, 2006.

[3] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[4] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click